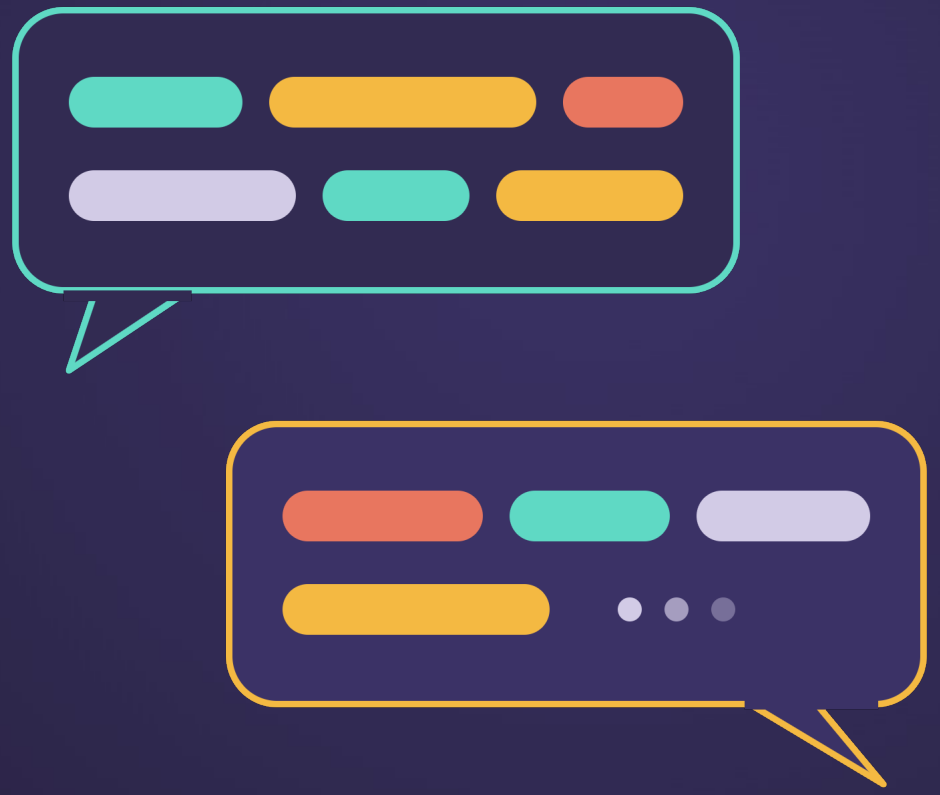
WEEK 4 · SATURDAY

How machines read language

From counting words to understanding meaning.



AI for Beginners · Lesson 4, Part A (NLP)

TODAY'S PLAN

From messy sentences to numbers a model can use

LAST WEEK IN ONE LINE

Pick the **right tool**, then judge it with the **right metric**.

Today the data is not a tidy table. It is language.

1. **Words into numbers**
   Three ways: IDs, counts, meaning. Then sentiment.
2. **Predicting the next word**
   Bigrams, trigrams and n-grams.
3. **Similarity and meaning**
   Text similarity and embeddings.
4. **Before attention**
   How models read in order, and where that breaks.

NATURAL LANGUAGE PROCESSING

NLP: teaching computers human language

Translation
English to Amharic, Swahili, French

Sentiment
Is this review happy or angry?

Search
Find pages that match what you mean

Chatbots
Answer questions, write, explain

Summaries
Shrink a long article to 3 lines

Speech
Voice to text and text to voice

All of them start with the same step: **turn words into numbers**.

WHY IT IS HARD

Language is messy

One word, many meanings

"I sat by the **bank**."

River bank or money bank?

Many words, one meaning

"big", "large", "huge", "enormous"

Different letters, nearly the same idea.

Tone and sarcasm

"Great, another meeting."

Positive words, negative feeling.

Context and order

"The dog bit the man" vs "The man bit the dog"

Same words, opposite story.

Also: typos, slang, mixed languages (like Sheng or Amharic with English).

Languages with **little online text** are hardest for AI.

THE BIG PICTURE

Three ways to turn words into numbers

1 Name tags

token IDs

pizza → 1052

Give each token an ID from the vocabulary list.

Good: simple. Every model starts here.

Weak: the number means nothing. 1052 is not "bigger" than 88.

2 Counting

bag of words, TF-IDF

good 2 | pizza 1 | cold 0

Count how often each word appears. TF-IDF weighs the counts.

Good: fast. You use it in today's notebook.

Weak: ignores word order and meaning.

3 Learned meaning

embeddings

pizza → 0.8 −0.3 0.5 ...

Each word becomes a list of numbers, learned from lots of text.

Good: close numbers mean close meanings.

Weak: needs lots of text to learn from.

Simplest to smartest: each way **fixes a weakness** of the one before.

STEP 1 · TOKENS

Chop text into tokens, then number them

The sentence	**I love learning AI!**
Tokens	I · love · learn · ing · AI · !
Token numbers	41 · 1052 · 3871 · 88 · 2207 · 3

A token is a piece of text: a word, part of a word, or a symbol.

Why word parts? "learning" = learn + ing, so new words are never a mystery.

Every model has a fixed **vocabulary list**. The numbers are just name tags with no meaning (example values).

STEP 2 · CLEAN-UP

Tidy the text before counting

Original	**The BEST pizza in town!!! Loved it**
1. lowercase	the best pizza in town!!! loved it
2. drop punctuation	the best pizza in town loved it
3. drop stop words	best pizza town loved
4. stem to the root	**best pizza town love**

⚠ **Careful:** "not" is often a stop word. Delete it and "not good" becomes "good".

STEP 3 · COUNT

Count the words: a bag of words

Give every word in the vocabulary its own column, then count.

Vocabulary →	good	bad	pizza	service	Label
R1 "good pizza good service"	2	0	1	1	positive
R2 "bad pizza bad service"	0	2	1	1	negative
R3 "bad service"	0	1	0	1	negative

Now it is a feature table. One row per review, one column per word. Same idea as feature engineering.

One-hot: one word, one 1. pizza is 0 0 1 0. **Bag of words** adds up a review's one-hots.

⚠ **Weak spot:** order is lost. "dog bit man" and "man bit dog" give the same row.

STEP 4 · WEIGH

TF-IDF: rare words say more

Bag of words counts every word equally. But "the" is in nearly every review, and "refund" is not.

Word	Appears in	Weight (how much it tells us)
the	98 of 100 reviews	tiny
pizza	40 of 100 reviews	medium
refund	3 of 100 reviews	**big**

weight = how often here × how rare overall

Short name: **TF-IDF**. Search engines used it for years.

YOUR FIRST NLP TASK

Sentiment analysis: happy or angry?

Review text → Clean → TF-IDF → Classifier → Positive or negative

Review	Model says	Confidence
"Loved the food, great service"	positive	
"Slow service and cold soup"	negative	
"Oh great, they lost my order"	positive?	

Sarcasm fools it. The words say "great", the meaning is angry.

You build this in today's notebook.

N-GRAMS

Slide a window over the words

The sentence	**I love learning AI**
Unigram 1 word	**I** · **love** · **learning** · **AI**
Bigram 2 words	**I love** · **love learning** · **learning AI**
Trigram 3 words	**I love learning** · **love learning AI**

N-gram = N words in a row. N is just the window size.

Why bother? Bigrams keep a little word order: "not good" stays together.

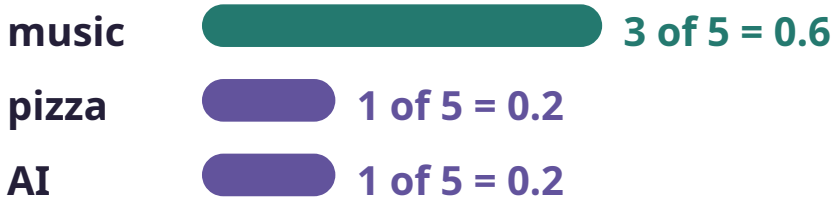
PREDICT THE NEXT WORD

Count, then divide

Our tiny corpus (5 sentences)

I love pizza
I love music
I love music
I love AI
you love music

After "love", what comes next?



P(music | love) = count("love music") ÷ count("love") = 3 ÷ 5 = 0.6

Read "|" as "given". P(music | love) means: chance of "music" given the last word was "love".

Your phone keyboard suggests words in a similar way.

WHERE N-GRAMS BREAK

Longer context, emptier table

Read 1 million words of text. You get at most 1 million word groups to count, whatever N is.

Unigram

10,000 boxes

one box per word

Filled: nearly all

Bigram

100 million boxes

10,000 × 10,000

Filled: at most 1%

Trigram

1 trillion boxes

10,000 × 10,000 × 10,000

Filled: at most 0.0001%

Empty box = no answer. Our tiny corpus never saw "you love pizza", so that trigram box is empty. Next slide: a real table.

The trap: too short and it forgets, too long and the boxes stay empty. More text helps, but the boxes grow faster.

Forgetting: "I grew up in Addis Ababa, so I speak fluent ___" needs a word far back. A trigram only sees "speak fluent".

WHERE N-GRAMS BREAK

A real table: most boxes are empty

Word before ↓ / Next word →	I	you	love	pizza	music	AI
I	0	0	**4**	0	0	0
you	0	0	**1**	0	0	0
love	0	0	0	**1**	**3**	**1**
pizza	0	0	0	0	0	0
music	0	0	0	0	0	0
AI	0	0	0	0	0	0

Bigram table

6 × 6 = 36 boxes

5 filled, 31 empty

Trigram table

36 rows × 6 = 216 boxes

4 filled, 212 empty

Empty box = no answer. "you love pizza" makes sense, but we only read "you love music".

More context = many more boxes, **same few sentences** to fill them.

WHERE N-GRAMS BREAK

How the tables are built

Bigram table

The clue is 1 word. Part of the table.

Clue ↓	love	pizza	music	AI	…
I	4	0	0	0	…
you	1	0	0	0	…
love	0	1	3	1	…
…	…	…	…	…	…

Row "love": pizza 1, music 3, AI 1.

Trigram table

The clue is 2 words. Part of the table.

Clue ↓	love	pizza	music	AI	…
I love	0	1	2	1	…
you love	0	0	1	0	…
love pizza	0	0	0	0	…
…	…	…	…	…	…

Row "I love": pizza 1, music 2, AI 1.

Row = the words you already have. **Column** = the next word. **Box** = how many times we saw it.

SIMILARITY OF TEXT

Are these two sentences alike?

Vocabulary →	i	love	pizza	pasta	stock	prices	fell
S1 "i love pizza"	1	1	1	0	0	0	0
S2 "i love pasta"	1	1	0	1	0	0	0
S3 "stock prices fell"	0	0	0	0	1	1	1

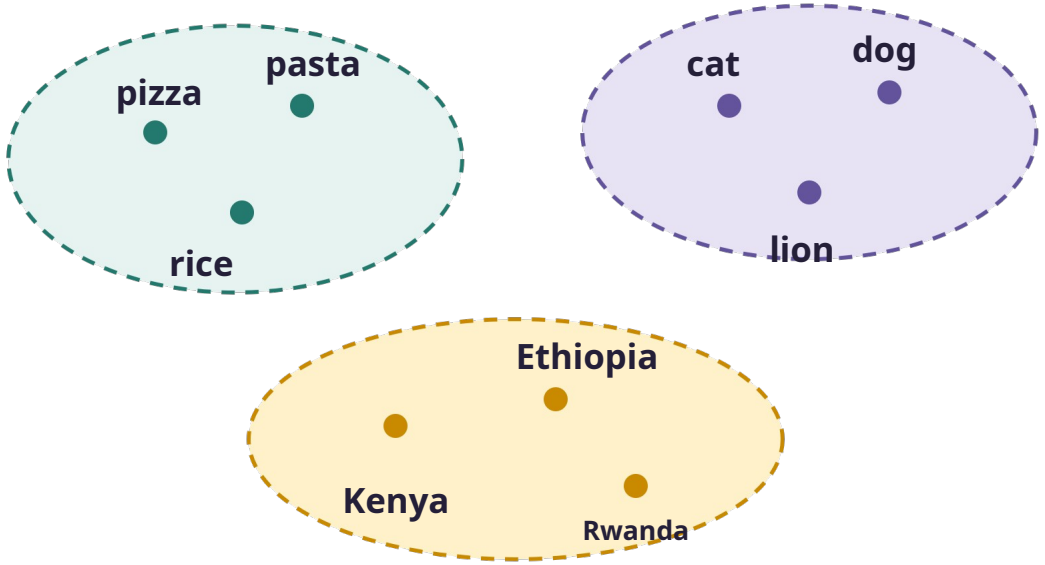
S1 vs S2: cosine 0.67
Share "i" and "love". Quite similar.

S1 vs S3: cosine 0
No words in common. Unrelated.

⚠ **Weak spot:** pizza and pasta are different columns, so the computer sees them as total strangers.

MEANING AS A MAP

Embeddings: words as points



Similar meaning = nearby points. The map is learned from lots of text.

In real models, each word has **hundreds of numbers**, not just two.

king − man + woman ≈ queen Directions on the map carry meaning.

PUTTING IT TOGETHER

Words into numbers: the whole ladder

Method	What the model sees	Knows meaning?	Good for
Token ID	pizza → 1052	No	First step of every model
Bag of words	good pizza → 1 0 1 0	No	Simple classifiers
TF-IDF	good pizza → 0.2 0 0.5 0	No, but rare words weigh more	Sentiment, search (your notebook)
Embeddings	pizza → 0.8 −0.3 0.5 …	Yes: similar words sit close	Modern language models
Contextual embeddings	bank → new numbers in each sentence	Yes, and it reads the context	Tomorrow: attention

Simple is not wrong. Counts and TF-IDF are fast, cheap and often good enough. Embeddings help when meaning matters.

BEFORE ATTENTION

Read word by word, with a memory

The → keys → to → the → cabinet → ? (are)

The memory is one small bundle of numbers, updated after every word and passed along.

Forgets

Early words fade. "keys" (plural) is far from the blank, so "is" can sneak in.

Slow

Word 50 must wait for words 1 to 49. Hard to speed up.

Bottleneck

The whole story squeezed into one small memory.

Tomorrow's question: what if every word could look back at every other word directly?

RECAP

Four things to remember

Text to numbers

IDs, counts or learned meaning. Careful with "not".

N-grams

Count word groups to predict the next word. Simple, but sparse and short-sighted.

Embeddings

Words as points on a map. Close points, close meanings.

Read in order

Older models forget early words and read slowly.

Notebook: clean text, build TF-IDF, train a sentiment model, make a bigram table.

Tomorrow: attention, GPT, and how machines hear and speak.

BEFORE TOMORROW

Try it yourself

1. Clean this review by hand: **"The Movie was NOT good!!!"** Which step is risky?

2. From **"I like tea. I like coffee. I like tea."** what is P(tea | like)?

3. Why are **"big"** and **"large"** far apart in bag of words, but close in embeddings?

Start the notebook too. We pick it up tomorrow.